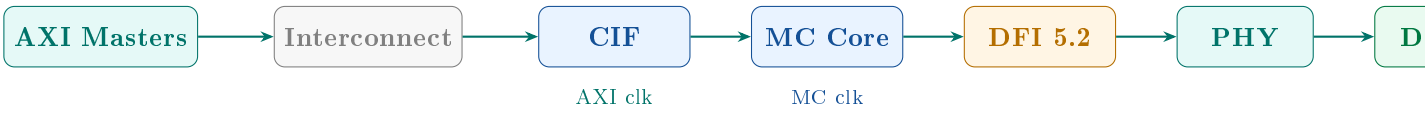
Reconfigurable Memory Controller

Design Specification

DDR5 · DFI 5.2 · AXI4



Standard	JEDEC JESD79-5 (DDR5), DFI 5.2, AXI4
Protocol	DDR through DDR5, LPDDR through LPDDR4 (runtime configurable)
Host IF	AXI4 (INCR bursts, 64-bit data bus, multiple outstanding IDs)
PHY IF	DFI 5.2 (1:1 / 1:2 / 1:4 frequency ratio)
Burst	BL16 default (BL/2 = 8 clock cycles per subchannel)
Banks	16 per rank (2 BG × 4 banks, DDR5 x8)

Contents

System Overview

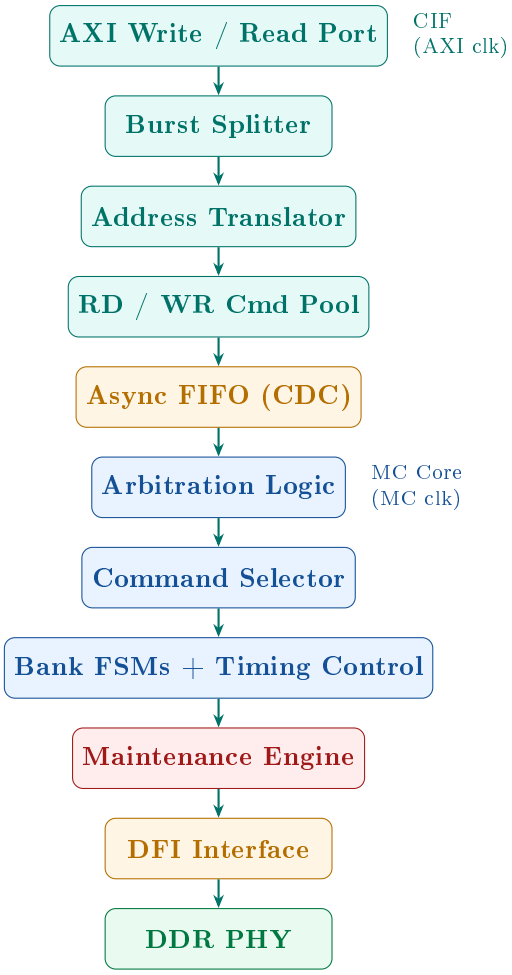
The Reconfigurable Memory Controller (RMC) is a universal DDR controller that interfaces with all major DDR and LPDDR generations without RTL changes. Protocol-specific behaviour — timing parameters, initialisation sequences, address mapping — is loaded from a software-programmable configuration register file at boot.

The RMC sits between an AXI4 system interconnect on the host side and a DFI-compliant PHY on the memory side. It is the *sole block responsible* for all DRAM protocol correctness: scheduling, timing enforcement, refresh, power management, and command generation. The PHY handles IO calibration and differential signalling. The scope of this document is the RMC block only.

Responsibility Boundaries

Block	Responsible For	Not Responsible For
CPU / L1–L3	Prefetching, cache management, coherency, speculation	DRAM timing, bank management, refresh
AXI Interconnect	Transaction routing, QoS arbitration between masters	Memory protocol, burst decomposition
CIF (Burst Splitter, Address Translator, Pools)	Receiving AXI bursts, page-boundary splitting, address translation, command queuing	Scheduling, timing, refresh, bank state
MC Core (Scheduler, Timing Control, BSMs)	Scheduling, timing enforcement, page policy, refresh, command generation	AXI protocol, PHY training, IO signalling
DFI Interface	Command and data transport between MC and PHY	Protocol decisions, scheduling
PHY	Training, read/write levelling, serialisation, electrical interface	DRAM protocol, scheduling

Full Datapath Flow



DDR5 Memory Hierarchy

Hierarchy

A physical address maps through the following levels, from coarsest to finest:

Channel → Subchannel → Rank → Bank Group → Bank → Row → Column → Byte Offset

Level	Definition
Channel	Top-level memory interface. The RMC supports one or two independent channels. Each channel has its own command and data bus.
Subchannel	DDR5 splits each 64-bit channel into two independent 32-bit subchannels (A and B). Each subchannel has its own DQS group and operates independently.
Rank	Group of DRAM chips sharing a chip-select signal (CS_n). All chips in a rank respond simultaneously. One DIMM = one rank; two DIMMs on one channel = two ranks.
Bank Group (BG)	DDR5 x8: 8 BGs per rank. DDR5 x16: 4 BGs per rank. Bank groups enable reduced CAS-to-CAS latency (tCCD_S) across groups vs. within the same group (tCCD_L).

Level	Definition
Bank	DDR5: 4 banks per BG (16 total per rank for x8). Each bank has its own row buffer (sense amplifiers). Only one row can be active per bank at a time.
Row	One activated page. DDR5 x8 8 Gb: 17-bit row address (131 072 rows). A row must be activated (ACT) before any RD/WR and precharged (PRE) before a different row can be opened.
Column	Position within an open row. DDR5 x8: 10-bit column address. Each column access fetches BL16 = 16 beats.
Byte Offset	Byte position within the burst. Not visible to the MC; handled by the Burst Splitter and data paths.

Burst Model

The RMC never fetches a single byte. Every DRAM access returns a full BL16 burst. The Burst Splitter and data path extract the requested bytes from the returned burst.

Quantity	Value
Burst Length (BL)	16 beats (BL16, DDR5 default)
Beat width (64-bit IF)	8 bytes
Full burst size (64-bit IF)	$16 \times 8 = 128$ bytes
Beat width per subchannel	4 bytes (32-bit subchannel)
Burst per subchannel	$16 \times 4 = 64$ bytes
Bus occupancy (BL/2)	8 MC clock cycles per burst

DIMM and Rank Configurations

Config	DIMMs	Ranks visible	Notes
1 DIMM	1	Rank 0 only	Subchannel mapping dominant
2 DIMM	2	Rank 0, Rank 1	Same channel; rank interleave available
2 Channel	1 per channel	Rank 0 per ch	Channel bit in address map
Full (4 DIMM dual rank)	4	2 per channel	Full address map: ch+subch+rank+BG+bank

Address Mapping

Design Principle

Address mapping is **fully runtime configurable**. No field widths are hardcoded in RTL. The bit boundaries for every DRAM address field are stored in Config Registers and read by the Address Translator at boot. This allows the same RTL to correctly handle DDR3, DDR4, DDR5, and LPDDR4 without modification.

Physical Address Decomposition

```
byte_addr[31:0] → {channel, subchannel, rank, BG, bank, row, col, offset}
```

Field widths depend on the installed configuration:

Field	Bits	Example	Notes
Byte offset	6	A[5:0]	$\log_2$(64 B cache line)
Subchannel	1	A[6]	0=SubchA, 1=SubchB
Column	10	A[16:7]	10-bit col for DDR5 x8
Bank	2	A[18:17]	4 banks per BG
Bank Group	3	A[21:19]	8 BGs for x4/x8
Rank	0–1	A[22]	0 bits = single rank
Channel	0–1	A[23]	0 bits = single channel
Row	17	A[31:15]	R[16:0] for 8 Gb DDR5

Configuration Cases

Config	Ch bits	Subch bits	Rank bits	BG bits	Bank bits
1 DIMM, 1 channel	0	1	0	3	2
2 DIMM, same ch	0	1	1	3	2
2 channel, 1 rank	1	1	0	3	2
Full (2ch, 2rank)	1	1	1	3	2

Interleaving Policy

Cache-line interleaving at 64-byte granularity is the recommended policy. Odd/even address splitting is explicitly rejected as it destroys spatial locality within a cache line.

Policy	Description	Decision
Odd/even address split	Alternate per byte address	**Rejected** — destroys locality
64-byte cache-line interleave	Alternate per 64 B block	**Adopted** — preserves locality

Client Interface (CIF)

The CIF terminates the AXI4 subordinate port and converts burst transactions into normalised RD_CMD / WR_CMD packets. It has no knowledge of DRAM timing, bank state, or refresh. It operates entirely in the AXI clock.

AXI Feature Decisions

Feature	Decision	Rationale
Burst type	INCR only	Covers all standard memory traffic. WRAP adds splitter complexity. FIXED not useful.
Data bus width	64-bit	Sufficient for target DDR generations.

Feature	Decision	Rationale
Outstanding IDs	Multiple	Required to hide DRAM latency. ROB handles out-of-order returns.
Narrow transfers	Not supported	Adds byte-lane steering complexity. Full bus width assumed.
Unaligned transfers	Not supported	High cost, low practical gain. Aligned bursts expected.
Exclusive access	Not supported	Requires reservation table. Out of scope.
QoS signals	Not supported	Scheduler complexity not justified at this stage.

Burst Splitter

Converts an incoming AXI burst into one or more memory commands, each confined to a single DRAM row and aligned to the burst length.

Stage 1 — Page Boundary Split. Checks whether the AXI burst crosses a DRAM row boundary. The row size is derived from the configured column width and bus width:

$$\text{ROW\_BYTES} = 2^{C} \times \text{BUS\_WIDTH\_BYTES}$$

A burst starting at address A crosses a boundary if:

$$A + \text{LEN\_BYTES} > \left\lfloor \frac{A}{\text{ROW\_BYTES}} \right\rfloor \times \text{ROW\_BYTES} + \text{ROW\_BYTES}$$

If so, the burst is split into sub-transactions, each confined to one row.

Stage 2 — Burst Length Alignment. Each sub-transaction is further split to fit within a single memory command of size:

$$\text{CMD\_BYTES} = \text{BL} \times \text{BUS\_WIDTH\_BYTES}$$

The number of commands generated for N_{bytes} bytes at address A' is:

$$N_{\text{cmds}} = \left\lceil \frac{(A' \bmod \text{CMD\_BYTES}) + N_{\text{bytes}}}{\text{CMD\_BYTES}} \right\rceil$$

Both stages are individually enable/disable at compile time via `STAGE1_EN` and `STAGE2_EN` flags.

Address Translator

Converts the burst-aligned byte address into the DRAM tuple {rank, BG, bank, row, col} using the configurable bit-field map from Config Registers. Packs the result with AXI metadata into a complete RD_CMD or WR_CMD packet.

Stage	Function
Bit Field Extractor	Slices `byte_addr[31:0]` into rank, BG, bank, row, col using programmed bit widths from `addr_map_mode`, `col_bits`, `row_bits`, `bank_bits`. Fully register-programmable; supports DDR3 through DDR5 without RTL change.

Stage	Function
CMD Builder	Assembles extracted DRAM fields with AXI metadata (AXID, tag, byte-enable mask) into the final RD_CMD / WR_CMD packet forwarded to the command pool.

Write Path

Incoming AXI write burst → Burst Splitter → Address Translator → Write Command Pool. Data travels with the command descriptor through the pool and across the CDC FIFO into the DRAM.

Read Path

Incoming AXI read burst → Burst Splitter → Address Translator → Read Command Pool → CDC FIFO → DRAM scheduler. Returning data follows the reverse path: Read Data Path → Merge Logic → Reorder Buffer → AXI Read Port.

Merge Logic

Reassembles fragments from split DRAM read transactions into complete burst responses before handing off to the Reorder Buffer.

Sub-block	Function
Tag Decoder	Extracts AXID, orig_id, frag_idx, frag_total from each returning {tag, data} beat.
Fragment Buffer	Stores each arriving beat at slot `[axid][frag_idx]`. Writes are non-blocking; fragments arrive in any order.
Reassembly Tracker	Per-AXID counters: `frag_count` (arrived) and `expected` (total, latched from `frag_total` on first fragment). Asserts `burst_complete` when equal.
Output Mux	Reads fragments in `frag_idx` order 0…`frag_total`−1, concatenates into a single burst, forwards to ROB. Single-fragment transactions bypass the buffer entirely.

Reorder Buffer (ROB)

Restores AXI read-response order after DRAM- reordering. Operates entirely in the AXI clock (CDC boundary is upstream).

Tag scheme: composite tag {AXID, seqnum} where seqnum is a per-AXID strictly-incrementing counter. Globally unique for all in-flight requests.

Property	Guarantee
Same-AXID ordering	Responses retire in strict seqnum order. HEAD pointer advances only when the next slot is valid.
Cross-AXID isolation	Per-AXID queues and HEAD pointers are independent. A stalled AXID never blocks another.
Tag uniqueness	{AXID, seqnum} is globally unique. seqnum is per-AXID and strictly increasing.

Property	Guarantee
No CDC inside ROB	The CDC boundary sits outside the ROB. The ROB operates on a single client clock, simplifying timing closure.

MC Core

The MC Core owns the clock- crossing. The Async FIFO bridges the AXI clock and the MC clock (running at half the DDR data rate). Commands cross as normalised WR_CMD or RD_CMD packets carrying address, data, byte-enable mask, and transaction tag.

Clock Domains

The Async FIFO CDC bar is the **sole** crossing point between the AXI clock and the MC clock. All blocks upstream of it (CIF, Burst Splitter, Address Translator, ROB, Merge Logic) run on the AXI clock. All blocks downstream (Bank FSMs, Timing Control, Scheduler, Maintenance Engine, DFI) run on the MC clock.

Config Registers

All runtime-programmable parameters for the entire RMC. Written via AXI4-Lite CSR path before `INIT_KICK`. May be updated at runtime only while the controller is idle.

Register Group	Width	Contents
Timing (JEDEC)	5–12b each	tRCD, tRP, tRAS, tRC, tWR, tRTP, tCCD_L, tCCD_S, tCCD_L_WR, tCCD_L_WR2, tWTR_L, tWTR_S, tRRD_L, tRRD_S, tFAW, tRFC1, tRFCsb, tREFI, tMRD, tXP, tXS, tCKSRE, tCKSRX, tDLLK
Latency	7b each	CL, CWL (= CL−2 in DDR5), PHY_WRLAT, PHY_RDLAT
Address Map	varies	addr_map_mode[2:0], col_bits, row_bits, bank_bits, bg_bits, rank_bits, ch_bits
Refresh	3b	REF_MODE: 00=REFab, 01=REFsb, 10=FGR-2x, 11=FGR-4x
Page Policy	2b	PAGE_POLICY: 00=Open, 01=Closed, 10=Adaptive
RFM (DDR5)	8–4b	RAAIMT, RAAMMT, RAADec
Arbitration	8b each	AGE_THR1, AGE_THR2, WR_HIGH_WM, WR_LOW_WM
DFI	2b	FREQ_RATIO: 1:1 / 1:2 / 1:4
Protocol Select	3b	DDR_VER: selects DDR2/3/4/5 or LPDDR4/5 timing gate set
Control	1b each	INIT_KICK, SOFT_RESET

Init FSM

Single instance. Triggered by `INIT_KICK`. Emits DFI commands directly, bypassing the Cmd Scheduler. Reads its command sequence and inter-command delays from Config Registers, making the same FSM correct for every supported DDR generation. Releases `init_done` on completion.

States (DDR5 sequence):

State	Action and Exit
IDLE	Wait for INIT_KICK. Assert RESET_n=0, CKE=0.
ASSERT_RESET	Hold RESET_n LOW. Count tINIT1 (200 µs).
CS_PRE_DEASSERT	Hold CS_n LOW for tINIT2 (10 ns) before deasserting RESET_n.
CS_POST_DEASSERT	RESET_n HIGH. CS_n LOW for tINIT3 (4 ms).
ODT_SETTLE	Raise CKE. Count tINIT4 (2 µs) for ODT and CK settle.
NOP_BURST	Issue ≥3 NOP/DES (tINIT5).
WAIT_XPR	Count tXPR = max(5 nCK, tRFC1+10 ns).
MPC_DLL_RESET	Issue MPC: DLL Reset. Required before ZQcal.
ZQCAL_START	Issue MPC: ZQCAL Start.
WAIT_tZQCAL	Count tZQCAL (1 µs).
ZQCAL_LATCH	Issue MPC: ZQCAL Latch.
WAIT_tZQLAT	Count tZQLAT (30 nCK).
MRW_BURST	Issue MRW to MR0, MR2, MR4, MR5, MR6, MR8, MR32–35, MR58. One MRW per tMRD (16 nCK).
TRAINING	Optional CA, CS, WL, read training via DFI handshake.
DONE	Assert `init_done`. Release Cmd Scheduler from reset.

DRAM Domain

All blocks in this section run on the MC clock. They are the sole arbiters of DRAM protocol correctness. The Cmd Scheduler issues one legal command per MC clock cycle to the DFI Interface.

Bank State Machines (BSM)

16 instances (one per bank). Each BSM independently tracks its bank through all DRAM states and owns all per-bank timers.

States:

State	Legal Inputs	Entry / Timer / Exit
IDLE	ACT, REFab, REFsb, PDE, SRE, RFM	Default. No timer. Entered from PRECHARGING (tRP), REFRESHING (tRFC), or at init.
ACTIVATING	(internal)	On ACT. tRCD countdown. → BANK_ACTIVE when tRCD expires.
BANK_ACTIVE	RD, RDA, WR, WRA, PRE, PREA	On tRCD expiry. tRAS loaded. PRE only after tRAS satisfied. Open row registered.
READING	BST	On RD/RDA. Bus occupied CL+BL/2 cycles. RDA: hidden PRE at tRTP. → BANK_ACTIVE (no AP) or PRECHARGING (RDA).

State	Legal Inputs	Entry/Timer/Exit
WRITING	BST	On WR/WRA. Bus occupied CWL+BL/2 cycles. WRA: hidden PRE at CWL+BL/2. → BANK_ACTIVE (no AP) or PRECHARGING (WRA).
PRECHARGING	(none)	On PRE/PREA or AP end. tRP countdown. → IDLE when tRP expires.
REFRESHING_AB	(none)	REFab. All 16 banks simultaneously. tRFC1 countdown. → all IDLE.
REFRESHING_SB	(none)	REFsb. Target bank only. tRFCsb countdown. Other banks remain accessible.
POWER_DOWN	PDX	Precharge PD (from IDLE) or Active PD (from BANK_ACTIVE). tCKE minimum low time. → IDLE/BANK_ACTIVE after PDX+tXP.
SELF_REFRESH	SRX	All banks precharged first. tCKSRE after entry. tXS before commands on exit.
RFM_ACTIVE	(none)	On RFM cmd (RAA ≤ RAAIMT). tRFM = tRFC1. → IDLE when tRFM expires.

Per-bank timers: tRCD, tRAS, tRC, tRP, tWR, tRTP, tRFC1, tRFCsb, tXS, tXP, tMRD. Each is a down-counter loaded in the same cycle as the triggering command.

Timing Control

Not a state machine. A **combinational and registered timer array** that produces `cmd_ok[15:0][CMD_TYPES]` every MC clock cycle. The Scheduler may only issue a command when the corresponding `cmd_ok` bit is set.

Cross-bank constraints checked:

Constraint	Scope	Implementation
tRRD_L	Same BG	Timestamp of last ACT in BG. Block until $\Delta T \geq$ tRRD_L.
tRRD_S	Diff BG	Timestamp of last ACT globally. Block until $\Delta T \geq$ tRRD_S.
tFAW	All banks	4-entry ring buffer of ACT timestamps. New ACT blocked if oldest within tFAW.
tCCD_L	Same BG, RD/WR	Last CAS timestamp in BG. Enforces tCCD_L / tCCD_L_WR / tCCD_L_WR2.
tCCD_S	Diff BG	Last CAS timestamp globally. Enforces $\Delta T \geq$ BL/2 = 8 nCK.
tWTR_L	Same BG, WR→RD	Last write data end in BG. Enforces WL+BL/2+tWTR_L.

Constraint	Scope	Implementation
tWTR_S	Diff BG, WR→RD	Last write data end globally. Enforces WL+BL/2+tWTR_S.
tRTW	Any, RD→WR	Last RD cmd. Enforces RL+BL/2−WL+2+tWPRE.

Arbitration Logic

Runs every MC clock. Determines *which command class wins this cycle*. Does not select bank, row, or column. Outputs: `arb_winner[2:0]`, `pool_ptr`, `mode_state`, `ref_urgent`.

Priority (0 = highest):

Level	Winner	Condition	Effect
0	REF urgent	Credits ≥ 8 or $\Delta t >$ tREFI$_{max}$	Hard stall all RD/WR
1	REF nominal	tREFI countdown = 0	REF injected; RD/WR deferred
2	ZQcal	tZQCS interval expires	ZQCS after current REF
3	Read	RD pool non-empty, RD mode	BG-aware scheduling
4	Write	WR pool non-empty, WR mode	Watermark-controlled
5	Scrubber/MRR	Idle slot	Pre-emptible

Anti-starvation (age counters): Each pool entry carries an age counter incremented every MC cycle while unserved. Age ≥ `AGE_THR1` (default 64): intra-class HOL bypass. Age ≥ `AGE_THR2` (default 256): cross-class escalation. Reset to 0 on dispatch.

RD/WR mode switching (watermarks): Switch to WR mode when WR pool depth ≥ `WR_HIGH_WM` (default 32) or RD pool empty. Switch back when depth ≤ `WR_LOW_WM` (default 8) or WR pool empty. Hysteresis gap = 24 entries prevents thrashing.

REF credit tracker: Credits += 1 every tREFI. Credits −= 1 per REF issued. At >85°C, tREFI is halved (MR4 TUF polling). `ref_urgent` fires when credits ≥ 8.

Command Selector

Takes the winner class, applies all remaining timing constraints, enforces page policy, and emits the final command tuple {cmd, rank, BG, bank, row, col, ap, wr_partial}.

Page policy:

Policy	Row Hit	Row Miss / Empty
Open (default)	CAS only	Miss: PRE→ACT→CAS. Empty: ACT→CAS.
Closed	ACT+CAS	ACT→CAS (always precharges). Best for random workloads.

Policy	Hit	Miss/Empty
Adaptive	Auto	Hit rate tracked over 256-access window per bank. >70%: open. <30%: closed. Auto-tunes per bank.

Legal check matrix — all gates must be true before dispatch: bank_avail[b], gate_rcd[b], gate_rp[b], gate_ras[b], gate_ccd_l, gate_ccd_s, gate_rrd_l, gate_rrd_s, gate_faw, gate_wtr_l, gate_wtr_s, gate_rtw, gate_rfc[r], gate_rfcpb[r][b], gate_zqcs, gate_mrd. Any false ⇒ NOP issued that cycle.

Command sequences emitted:

Decision	Sequence
Row Hit	CAS (RD/WR)
Row Miss	PRE $\xrightarrow{\text{tRP}}$ ACT $\xrightarrow{\text{tRCD}}$ CAS
Row Empty	ACT $\xrightarrow{\text{tRCD}}$ CAS
REFab	REFab $\xrightarrow{\text{tRFC1}}$ NOPs (all banks blocked)
REFsb	REFsb(bank) $\xrightarrow{\text{tRFCsb}}$ NOP (target bank only)
ZQCS	MPC ZQCAL Start $\xrightarrow{\text{tZQCAL}}$ MPC ZQCAL Latch $\xrightarrow{\text{tZQLAT}}$ ready
MRW	MRW $\xrightarrow{\text{tMRD}}$ next command

Maintenance Engine

Four independent sub-FSMs posting requests to the Scheduler via a priority request bus.

Refresh Scheduler

States: IDLE → REF_DUE → WAIT_BANKS_IDLE → ISSUE_REF → WAIT_tRFC → DONE → IDLE.

REFsb path: WAIT_TARGET_BANK_IDLE → ISSUE_REFsb → WAIT_tRFCsb instead of the REFab path.

Leaky-bucket counter increments every tREFI; decrements on each REF issued; `ref_urgent` at count = 9 (8 postponements exhausted).

ZQcal FSM

States: IDLE → WAIT_IDLE → ISSUE_ZQCAL_START → WAIT_tZQCAL → ISSUE_ZQCAL_LATCH → WAIT_tZQLAT → DONE → IDLE.

Triggered by periodic tZQCS timer or `ZQCAL_TRIG` bit.

RFM FSM (Rowhammer, DDR5)

States: IDLE → MONITOR_RAA → RFM_REQUEST → WAIT_ISSUE → WAIT_tRFM → UPDATE_RAA → MONITOR_RAA.

RAA[b] += 1 per ACT to bank b. RAA[b] −= `RAADec` per REF. RFM request posted when RAA[b] ≤ `RAAIMT`.

Power Management FSM

Normal → PD_ENTRY_CHECK → PRECHARGE_PD or ACTIVE_PD → PDX_WAIT_tXP → Normal.

SR branch: Normal → SR_ENTRY → WAIT_tCKSRE → SELF_REFRESHING → SR_EXIT → WAIT_tCKSRX → WAIT_tXS → WAIT_tDLLK → Normal.

Data Paths and DFI

Write Data Path: BL16-aligned write buffer → DM/ECC generation → Write CRC (CRC-8, MR8 OP[6]=1) → WL pipeline alignment → DFI write timing insertion.

Read Data Path: DFI read capture FIFO → Read CRC/ECC checker → read latency counter → tag association → return to CIF.

Write CRC Error FSM: NORMAL → CRC_ERROR → RETRY_WRITE → AUTO_DISABLE. Monitors `dfi_alert_n`. Auto-disables after n retries (threshold from MR51/52).

ECS FSM (DDR5): IDLE → ECS_ACTIVE → WAIT_tECS → IDLE. Triggered by MR14. DRAM scrubs on-die ECC during tECS window.

DFI 5.2 Interface

Signal	Dir	Description
`dfi_address[13:0]`	MC→PHY	CA bus. 2-cycle encoding for DDR5 ACT.
`dfi_cs_n[1:0]`	MC→PHY	Chip select per rank.
`dfi_cke[1:0]`	MC→PHY	Clock Enable per rank.
`dfi_odt[1:0]`	MC→PHY	ODT control per rank.
`dfi_reset_n`	MC→PHY	DRAM RESET_n. Driven by Init FSM.
`dfi_act_n`	MC→PHY	Activate indicator.
`dfi_bank[1:0]`	MC→PHY	Bank address.
`dfi_bg[2:0]`	MC→PHY	Bank group (BG[2:0] x4/x8; BG[1:0] x16).
`dfi_wrdata[127:0]`	MC→PHY	Write data (BL16 × DQ8).
`dfi_wrdata_mask[15:0]`	MC→PHY	Write data mask per byte.
`dfi_wrdata_en`	MC→PHY	Asserted `PHY_WRLAT` cycles before data.
`dfi_wrdata_cs_n`	MC→PHY	Per-DRAM write CS (DDR5 PDA).
`dfi_parity_in`	MC→PHY	CA parity (DDR5).
`dfi_rddata[127:0]`	PHY→MC	Captured read data.
`dfi_rddata_valid`	PHY→MC	Read data valid.
`dfi_rddata_dnv`	PHY→MC	Data-not-valid error.
`dfi_alert_n`	PHY→MC	DRAM alert (CRC error, CA parity).
`dfi_ctrlupd_req/ack`	MC↔PHY	Controller PHY update handshake.
`dfi_phyupd_req/ack`	PHY↔MC	PHY pause request handshake.
`dfi_init_start/complete`	MC↔PHY	Init sequence handshake.
`dfi_freq_ratio[1:0]`	MC→PHY	00=1:1, 01=1:2, 10=1:4.

Open Architecture Questions (ADRs)

The following decisions are pending formalization. Each is assigned an Architecture Decision Record (ADR) number.

ADR	Topic	Open Question
ADR-0001	Command Format	Final RD_CMD/WR_CMD packet field layout and widths.
ADR-0002	Address Mapping	v0 complete. Needs formalization of XOR hashing option and multi-channel MC assignment.
ADR-0003	CDC Contract	Exact packet format, flow control, and backpressure semantics across Async FIFO.
ADR-0004	Queue Architecture	Depth of RD/WR Cmd Pools, tag structure, allocation policy, retirement ordering.
ADR-0005	Command Lifecycle	Full flow: allocation → dispatch → issue → completion → retirement.
ADR-0006	Timing Ownership	Who owns per-bank timers? Who loads them? Cross-block communication protocol between Timing Control and BSMs.

Proposals Under Discussion

The items below are design proposals explored during architecture discussion. None are finalized decisions. Each is listed with its motivation, current thinking, and open questions.

Dual MC Core Architecture

Proposal: Replace one large monolithic MC with two smaller independent MC cores, each handling a subset of channels/subchannels.

Motivation: Natural parallelism between DDR5 subchannels. Smaller scheduler, timing database, and command engine per core. Potentially easier verification.

Proposed assignment:

- MC0: Channel 0A + Channel 1B
- MC1: Channel 1A + Channel 0B

This ensures both MCs remain active regardless of whether 1, 2, or 4 DIMMs are installed.

Open questions: Global refresh coordination. Shared maintenance operations. Cross-MC address interleave policy.

Status: Not finalized. Further evaluation required.

Burst Reuse Buffer

Proposal: A small buffer (depth ≈ 2) between the DRAM and the CIF that retains unrequested bytes from a BL16 burst.

Motivation: A BL16 burst returns 64 B per subchannel. If only 32 B are needed, the remaining 32 B may satisfy the next sequential request without a new DRAM access.

Exploits: Spatial locality and sequential access patterns.

Open questions: Buffer depth, replacement policy, interaction with ROB tag tracking.

Status: Not implemented. Proposed for future evaluation.

Row Hit Predictor

Proposal: A per-bank predictor that outputs KEEP_OPEN or AUTO_PRECHARGE to guide the page policy engine.

Motivation: Static open/closed page policy is suboptimal for mixed workloads. An adaptive predictor could reduce wasted PRE+ACT overhead without sacrificing row-hit rate.

Options: Static threshold (current Adaptive policy), dynamic saturating counter per bank, or a more sophisticated history-based predictor.

Status: Initial RTL uses open-page with configurable adaptive threshold. Full predictor deferred.

XOR Address Hashing

Proposal: XOR bits from the upper row address into the BG and bank fields to reduce bank conflicts on regular stride patterns.

Example:

$$\text{BG0} = A_{14} \oplus A_{18}, \quad \text{BG1} = A_{15} \oplus A_{19}$$

Motivation: Stride-aligned memory access patterns (e.g., matrix operations) map disproportionately to the same bank group. XOR hashing spreads these across BGs.

Status: Not part of v1 address map. Marked for ADR-0002 follow-up.

Runtime DIMM Discovery

Proposal: Boot-time DIMM detection logic that automatically configures address mapping field widths based on detected DIMM count, rank count, and DRAM geometry (SPD readout).

Motivation: Currently, Config Registers are written by software. Automatic discovery removes the need for OS-level memory probing.

Status: Out of scope for current phase. Requires SPD I^2C reader integration.

Locked Design Decisions

Decision	Rationale
Burst-based, not byte-based	DRAM always returns BL16. Byte extraction is a datapath concern, not a scheduling concern.
Address mapping is runtime configurable	DIMM count, rank count, DRAM density, and BG count all vary. Hardcoding field widths is incompatible with a universal controller.
Cache-line (64 B) interleaving	Odd/even address splitting destroys spatial locality within a cache line. 64 B interleaving preserves it.
Open page first	Sequential and localised workloads are common. Open page delivers row hits without PRE+ACT overhead. Adaptive policy added on top.
Prefetching not owned by MC	Prefetch is a CPU-side concern. The MC responds to requests; it does not predict them.
Burst Splitter before MC	MC Core sees only well-formed, single-row, BL-aligned commands. This simplifies the scheduler significantly.
Address Translator before MC	MC Core operates on {rank, BG, bank, row, col} tuples only. No byte address arithmetic inside the scheduler.

Decision	Rationale
Single CDC crossing point	One Async FIFO between AXI and MC clocks. Simplifies timing closure and eliminates multi- synchronisation complexity.
AXI INCR bursts only	Covers all standard memory traffic. WRAP and FIXED burst types add splitter complexity with no practical benefit for DRAM access.